

Notebook: logistic_regression/logistic_regression_on_real_data.ipynb

[--- MARKDOWN CELL ---]

Logistic Regression on Real Data

This notebook applies our from-scratch logistic regression to multiple real-world datasets and compares performance.

[--- CODE CELL ---]

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelBinarizer
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.linear_model import LogisticRegression as SklearnLR
```

[--- CODE CELL ---]

```
np.random.seed(42)
```

[--- MARKDOWN CELL ---]

Logistic Regression Class (From Scratch)

[--- CODE CELL ---]

```
class LogisticRegressionScratch:
    """Logistic Regression from scratch using gradient descent."""
    def __init__(self, lr=0.01, epochs=1000):
        self.lr = lr
        self.epochs = epochs
        self.w = None
        self.b = None
        self.cost_history = []

    def sigmoid(self, z):
        z = np.clip(z, -500, 500)
        return 1 / (1 + np.exp(-z))

    def compute_cost(self, X, y):
        m = len(y)
        z = np.dot(X, self.w) + self.b
        y_pred = self.sigmoid(z)
        epsilon = 1e-15
        y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
        cost = (-1/m) * np.sum(y * np.log(y_pred) + (1 - y) * np.log(1 - y_pred))
        return cost

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.w = np.zeros(n_features)
        self.b = 0
        self.cost_history = []
        for _ in range(self.epochs):
            z = np.dot(X, self.w) + self.b
            y_pred = self.sigmoid(z)
            error = y_pred - y
            dw = (1/n_samples) * np.dot(X.T, error)
            db = (1/n_samples) * np.sum(error)
            self.w -= self.lr * dw
            self.b -= self.lr * db
            cost = self.compute_cost(X, y)
```

```

        self.cost_history.append(cost)
    return self

    def predict(self, X, threshold=0.5):
        z = np.dot(X, self.w) + self.b
        y_pred_prob = self.sigmoid(z)
        return (y_pred_prob >= threshold).astype(int)

    def predict_proba(self, X):
        z = np.dot(X, self.w) + self.b
        return self.sigmoid(z)

    def score(self, X, y):
        y_pred = self.predict(X)
        return accuracy_score(y, y_pred)

```

[--- MARKDOWN CELL ---]

Dataset 1: Breast Cancer

[--- CODE CELL ---]

```

# Load and prepare data
df_bc = pd.read_csv('../datasets/breast_cancer.csv')
X_bc, y_bc = df_bc.drop('target', axis=1).values, df_bc['target'].values
X_train, X_test, y_train, y_test = train_test_split(X_bc, y_bc, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train from-scratch model
lr_scratch = LogisticRegressionScratch(lr=0.1, epochs=1000)
lr_scratch.fit(X_train_scaled, y_train)

# Train sklearn model
lr_sklearn = SklearnLR(max_iter=1000, random_state=42)
lr_sklearn.fit(X_train_scaled, y_train)

print("Breast Cancer Dataset:")
print(f"  From Scratch Accuracy: {lr_scratch.score(X_test_scaled, y_test):.4f}")
print(f"  Sklearn Accuracy: {lr_sklearn.score(X_test_scaled, y_test):.4f}")

```

[--- CODE CELL ---]

```

# Plot training curve
plt.figure(figsize=(10, 5))
plt.plot(lr_scratch.cost_history)
plt.xlabel("Epochs")
plt.ylabel("Cost")
plt.title("Training Loss Curve - Breast Cancer Dataset")
plt.grid(True, alpha=0.3)
plt.show()

```

[--- MARKDOWN CELL ---]

Dataset 2: Iris (Binary - Setosa vs Others)

[--- CODE CELL ---]

```

# Load Iris and make it binary
df_iris = pd.read_csv('../datasets/iris.csv')
X_iris, y_iris = df_iris.drop('target', axis=1).values, df_iris['target'].values
y_iris_binary = (y_iris == 0).astype(int) # Setosa vs not Setosa
X_train, X_test, y_train, y_test = train_test_split(X_iris, y_iris_binary, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

```
# Train models
lr_scratch = LogisticRegressionScratch(lr=0.1, epochs=1000)
lr_scratch.fit(X_train_scaled, y_train)
lr_sklearn = SklearnLR(max_iter=1000, random_state=42)
lr_sklearn.fit(X_train_scaled, y_train)
print("Iris Dataset (Binary):")
print(f"  From Scratch Accuracy: {lr_scratch.score(X_test_scaled, y_test):.4f}")
print(f"  Sklearn Accuracy: {lr_sklearn.score(X_test_scaled, y_test):.4f}")
```

[--- MARKDOWN CELL ---]

Comparison Summary

[--- CODE CELL ---]

```
# Create comparison bar chart
datasets = ['Breast Cancer', 'Iris (Binary)']
scratch_scores = [0.9561, 1.0000] # Example scores
sklearn_scores = [0.9649, 1.0000] # Example scores
x = np.arange(len(datasets))
width = 0.35
fig, ax = plt.subplots(figsize=(10, 6))
bars1 = ax.bar(x - width/2, scratch_scores, width, label='From Scratch', color='steelblue')
bars2 = ax.bar(x + width/2, sklearn_scores, width, label='Sklearn', color='coral')
ax.set_ylabel('Accuracy')
ax.set_title('Logistic Regression: From Scratch vs Sklearn')
ax.set_xticks(x)
ax.set_xticklabels(datasets)
ax.legend()
ax.set_ylim(0.9, 1.05)
ax.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()
```